Grasp — A Structured Learning App

Project Proposal

Team	Course	Date	Version
Null Pointers	ECE 452	June 05, 2026	1.0

1. Introduction

Motivation and Project Overview

Learning something new should feel exciting, but for many people, the hardest part is simply figuring out where to begin. Searching online often leads to an overwhelming amount of information, leaving learners unsure of which concepts matter most or how they relate to one another. Without a clear path to follow, many feel lost before they even have a chance to start learning.

Our proposed solution to this problem is Grasp, a mobile app that breaks down dense topics into structured learning paths that are easy to follow. Users enter a topic, and the app splits it into smaller subtopics, which are then arranged in a logical order to create a learning path. Each subtopic contains a short introduction or excerpt to help users grasp the basic idea. In addition, the app allows users to save content and access additional resources such as articles and books, along with an AI learning assistant, to expand their knowledge. A visual mock-up of the app's interface is shown in Figure 1.

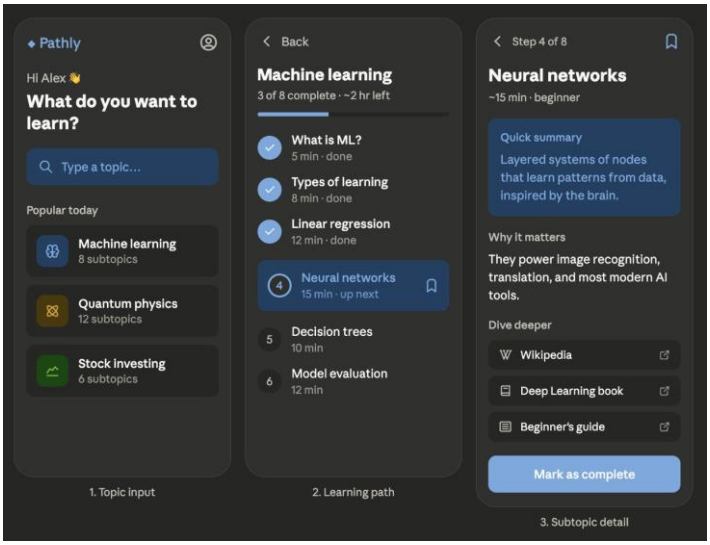


Figure 1: App mock-up

Project Interest and Significance

What makes this project particularly interesting is its ability to make the learning process simple, structured, and intuitive. Tools like ChatGPT can quickly provide information on almost any topic, but the information is typically presented in a conversational format. Trying to learn a topic through one long, messy stream of information can make it hard for users to keep track of concepts and determine what to learn next. In contrast, our app organizes topics into clear paths and sections (see Figure 2). This helps users follow the progression and understand how different ideas connect as they move through the topic. In addition to covering the basics, the app also allows users to explore topics in more depth using the provided resources. Users will never lose track of their learning progress, even if they get sidetracked or switch

to a different topic entirely. Together, these features create a learning environment that is simple to navigate yet still flexible toward each user's needs.

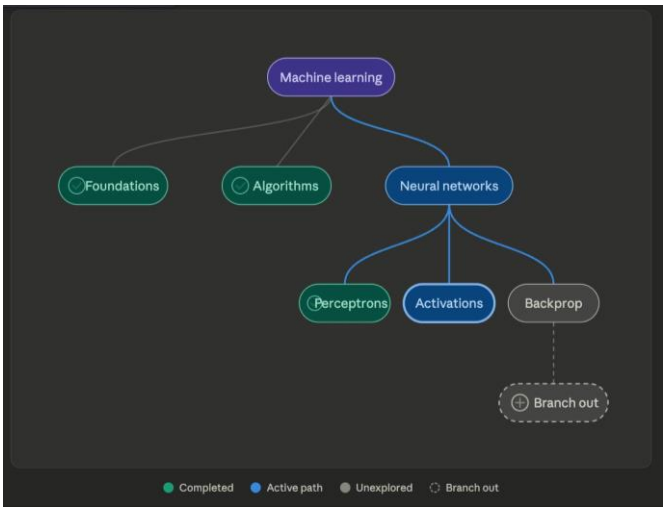


Figure 2: Example learning path

Why Mobile?

Mobile learning is well suited to how people already use their phones throughout the day. Generally, this includes short periods of use during commutes, breaks, or random moments of boredom or curiosity. The app is designed for these sorts of usage habits, as it breaks content down into small sections that can be completed quickly without needing to focus for too long. Users can save their progress, resume where they left off, and switch between topics without losing their place. Because phones are portable, the learning is always accessible, so users can use the app whenever they have a few minutes to spare. This keeps learning continuous through short and frequent sessions, which helps form a learning habit.

2. User Scenarios

Scenario 1

Jordan is a 1A UW ECE student who just moved into his first apartment and wants to stop relying on Lazeez and Campus Pizza. He's tried watching cooking videos online but ends up jumping between random videos due to his short TikTok attention span, and has no sense of what skills to build first. Jordan opens the app and types "cooking for beginners". The app then generates a structured learning tree for the topic. The tree breaks the subject into subtopics such as Kitchen Basics, Knife Skills, and Simple Techniques, each with an estimated time to complete. Jordan selects the tree view to see how the concepts connect to each other. The roadmap appears with all subtopics clearly laid out.

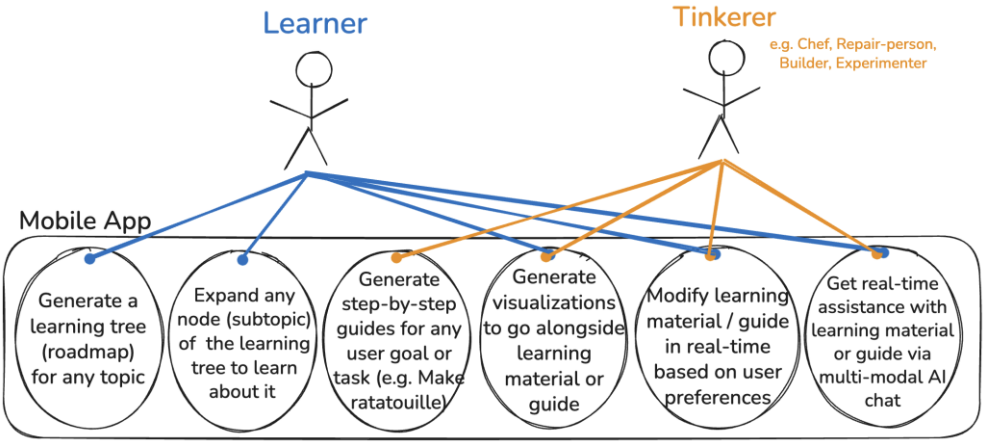
Scenario 2

Inside the roadmap, Jordan taps into the Knife Skills subtopic and reads through the subsequently generated content. A section on dicing onions is unclear, and Jordan isn't sure how to make even cuts without the onion falling apart. He highlights the sentence and opens the inline AI chat to ask about it. The app responds to his question, and Jordan follows up asking about safety techniques, as his fingertips are required for an upcoming technical interview. Jordan marks the subtopic complete and the progress bar visibly advances.

Scenario 3

Moving on to try and cook an actual meal, Jordan switches to Tinkerer mode and types "make a breakfast omelette." The app generates a step by step guide with checkboxes. The process is broken up into actionable and easy to follow segments. Partway through, Jordan isn't sure the eggs look right in the pan, so he takes a photo and asks through the built in chat whether the stove is at the correct temperature. The app suggests an adjustment after viewing the photo. Jordan checks off each step as he goes and the checklist updates to reflect his progress.

3. Use Case Diagram



4. Functional Requirements

1. Topic & Goal Entry

1.1) The app shall let a user enter a topic to learn (Learner) or a task to accomplish (Tinkerer)

2. Learning Mode

2.1) The app shall generate a roadmap that breaks a topic into subtopics.
2.2) The app shall let a user view the roadmap as a visual tree or a standard list.
2.3) The app shall open a subtopic's content when a user selects it.
2.4) The app shall let a user mark subtopics complete and show progress as node states in tree view or a progress bar in list view.

3. Tinkering Mode

3.1) The app shall generate a flat, ordered step-by-step guide for the user's task.
3.2) The app shall show progress as a step checklist with a linear progress bar

4. Content Generation

4.1) The app shall generate content in clickable sections/blocks that users can open to chat about (see 5).
4.2) The app shall generate the content based on their preferences (e.g. difficulty, length, format).

4.3) The app shall generate an estimated time to complete for each subtopic/step.
4.4) (Optional) The app shall generate visualizations to go along with the learning material or guide.
4.5) The app shall let a user manually edit any generated content

5. AI Chat Assistance

5.1) The app shall provide a multi-modal AI chat for help with the current material.
5.2) The app shall let a user select a section or highlight part of the content to ask about it.
5.3) The app shall answer a user's questions about the material in the chat.
5.4) The app shall update content on the user's request or suggest changes the user can accept.
5.5) The app shall save and let a user revisit conversation history for all chats.

6. Quizzes (Learn Mode, Optional)

6.1) The app shall optionally generate quizzes in multiple formats (e.g. MCQ, timed) for a subtopic/topic. 6.2) The app shall let a user interactively select, type, or upload a photo of their work to answer questions. 6.3) The app shall grade the user's answers with feedback and explanations
6.4) The app shall save quizzes and the user's answers.
6.5) The app shall let a user retake a quiz.
6.6) The app shall let a user compare answers of a retaken quiz with previous attempts.

7. Managing Topics & Guides

7.1) The app shall save a user's topics and guides.
7.2) The app shall let a user resume a saved topic or guide.
7.3) The app shall let a user remove a topic or guide.
7.4) The app shall let a user export a topic or guide for others to use
7.5) The app shall let a user import a topic or guide from others

8. Account (Optional)

8.1) The app shall let a user optionally create an account to sync their data across devices.

5. Non-Functional Requirements

1. Performance

1.1) The app should respond quickly to keep learners engaged. AI-generated learning paths should load within reasonable time (around 5 seconds).
1.2) Navigating between subtopics, saved paths, and resource links should feel instant with no noticeable lag or UI glitch.

2. Usability

2.1) The interface should be intuitive enough that a new user can generate their first learning path without any instructions or tutorials.
2.2) Since the app's users ranges between all kind of ages and backgrounds, the UI must be clean and uncluttered.

3. Reliability

3.1) The app should handle failures gracefully, as in, if the AI service is unavailable or the network drops mid-generation, the user should see a meaningful error message rather than a crash.
3.2) Previously saved paths must always be accessible, even when offline.

4. Security

4.1) If the app will require the user to make an account; hence, any account credentials, respective saved paths, and progress must be stored and transmitted securely.
4.2) Passwords must be hashed with industry-standard authentication.
4.3) API keys for the AI service must never be exposed on the client side.

5. Maintainability

5.1) The codebase should follow a specific GUI architecture (e.g MVC, MVP, MVVM).
5.2) Code should be well documented, so a new developer would understand the structure without needing a walkthrough.

6. Design Considerations

Human Values

- **Autonomy:** We empower users to direct their own learning by generating custom learning trees or guides for any topic. Allowing users to dynamically modify their guides ensures the curriculum adapts to their specific needs, rather than forcing them into a rigid, predefined structure.
- **Inclusivity:** To cater to users of all ages and educational backgrounds, the app features a clean, uncluttered UI. Furthermore, the multi-modal AI chat, which accepts both text and photos, supports diverse learning styles and helps those who might struggle with text-only queries.

Unintended Harms

- **Mitigated Feature:** We initially considered having the AI automatically advance user progress based on chat interactions. This risks causing frustration and a loss of control if concepts are marked complete prematurely. *Mitigation:* We explicitly designed requirements so that users must manually mark subtopics complete, restricting the AI to a purely assistive role.
- **Overlooked Feature:** Because users can input any task in Tinkerer mode, they could potentially request guides for dangerous activities (e.g., hazardous chemical experiments). We realized that strict AI safety guardrails and content filtering are critical to prevent the generation of harmful, real-world instructions.

Requirement Trade-offs and Design Decisions

Conflict 1: Generation Speed vs. Content Completeness

- **The Conflict: Performance (NFR 1.1)** requires AI learning paths to load within 5 seconds to keep learners engaged. However, generating a complete, deeply nested learning tree with excerpts, guides, and visualizations for an entire topic requires significant computational time, clashing with the need for immediate responsiveness.
- **Design Decision and Why:** We prioritized initial loading speed over upfront completeness. The app generates only the high-level roadmap structure upon the initial search to meet the 5-second constraint. Deeper text and instructional content are generated or fetched asynchronously only when a user expands a specific node. This provides immediate visual feedback to maintain engagement, balancing AI technical limitations with mobile usability expectations.

Conflict 2: Offline Reliability vs. Cloud-Dependent AI Interactivity

- **The Conflict: Reliability (NFR 3.2)** dictates that previously saved learning paths must be accessible offline. However, dynamic functional requirements such as generating new content (FR 4.1), updating material (FR 5.4), and real-time chat assistance (FR 5.1), they all rely entirely on continuous internet access and cloud-based AI APIs.

- **Design Decision and Why:** We prioritized reliable access to existing knowledge over full offline feature availability. The application locally caches the text, structure, and progress states of previously saved paths. When offline, users can read saved content, update Tinkerer checklists, and mark subtopics complete. Dynamic features like AI chat or expanding ungenerated nodes are temporarily disabled, gracefully displaying an error message (NFR 3.1). This preserves access to ongoing progress during offline periods (like commutes) while acknowledging network connectivity limits.

Conflict 3: UI Simplicity vs. Advanced Customization

- **The Conflict:** There is a conflict between the needs of Novice Learners (who need simplicity) and Power Users/Tinkerers (who want deep control). Usability (NFR 2.2) demands a clean, uncluttered UI to accommodate users of all ages and backgrounds. Conversely, functional requirements like letting users manually edit generated content (FR 4.5) require adding complex text editors, buttons, and visual indicators to the screen, which inherently clutters the interface.
- **Design Decision and Why:** We prioritized UI simplicity for the default user experience. Advanced customization features like manual editing will be tucked away behind secondary menus or a dedicated "Edit Mode" toggle. This ensures the default view remains a clean, focused reading environment for Novice Learners, while still providing the necessary tools for Tinkerers who intentionally seek them out.

7. Milestone Schedule

Week	Goal	Requirements
Setup & Design		
5	Project & repo setup + Design finalization	**FR:** 1.1 **NFR:** 2.1, 2.2, 5.1, 5.2
Core Development		
6	Core AI integration & subtopic generation	**FR:** 2.1, 3.1, 4.2, 4.3 **NFR:** 1.1, 4.3
7	Learning tree: base & interactions	**FR:** 2.2, 2.3, 2.4, 3.2 **NFR:** 1.2
8	Chatbot integration & content blocks	**FR:** 4.1, 5.1, 5.2, 5.3, 5.4
Advanced Features		
9	Save, progress, read time & export	**FR:** 4.3, 5.5, 7.1, 7.2, 7.3, 7.4, 7.5, 8.1* **NFR:** 3.2. 4.1, 4.2
10	UI polish, visualizations & manual editing	**FR:** 4.4, 4.5 **NFR:** 1.2, 2.1, 2.2
Testing & Launch		
11	Testing & bug fixes	**FR:** 6.1*, 6.2*, 6.3*, 6.4*, 6.5*, 6.6* **NFR:** 1.1, 3.1, 3.2, 4.1, 4.2, 4.3
12	Demo prep	*All FRs and NFRs validated end-to-end*

8. References

- Mobile Learning & Educational Apps
  - https://blog.duolingo.com/duolingo-score/

- AI in Education
  - https://openai.com/research
  - https://www.anthropic.com/research
  - https://www.edweek.org/technology/opinion-yes-schools-should-allow-students-to-use-ai/2026/05

- UI/UX & Usability
  - https://developer.android.com/design/ui/mobile/guides/foundations/system-bars
  - https://developer.android.com/design/ui/gallery/reading/abcmouse?hl=en&authuser=0
  - https://developer.apple.com/design/human-interface-guidelines

- Software Architecture
  - https://developer.android.com/topic/architecture
  - https://developer.apple.com/documentation

- Cognitive Load / Learning Science
  - https://www.learningscientists.org — evidence-based learning strategies
  - https://insightstobehavior.com/blog/managing-attention-in-the-classroom/

9. Attributions

- Figures 1 & 2 were html files generated by Claude.